



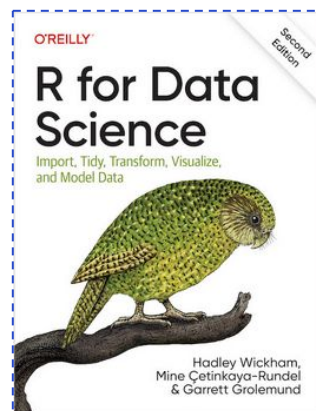
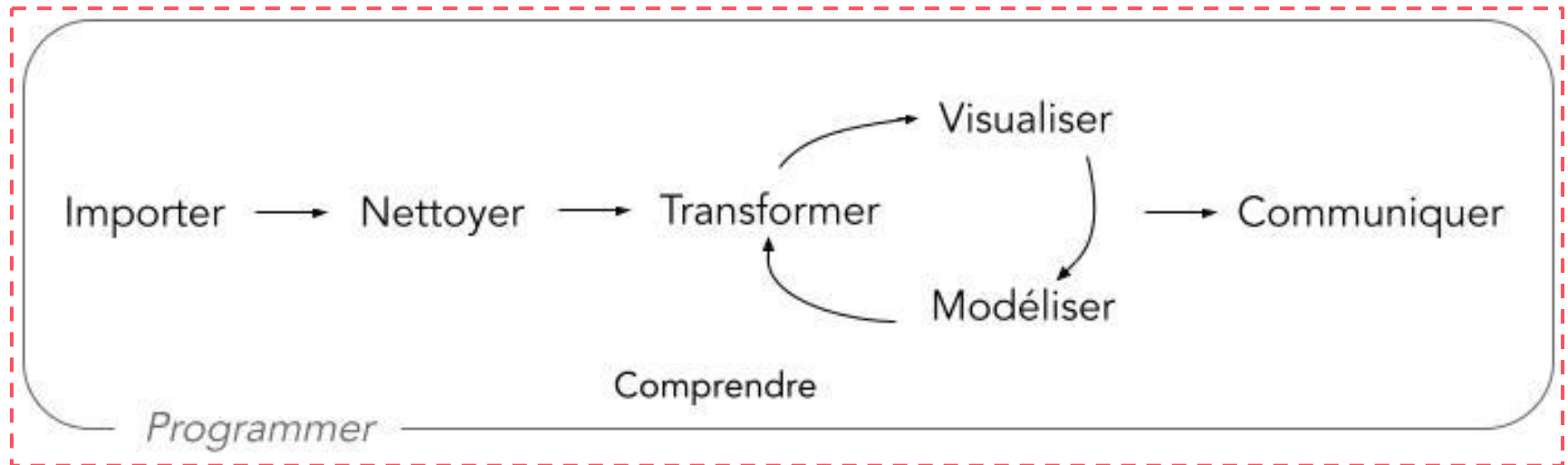
Communauté
d'**Agglomération** de
La Rochelle



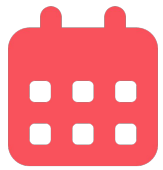
Introduction au langage de programmation R

25 juin 2026

Le programme de cette journée ; le schéma d'exploitation de la donnée

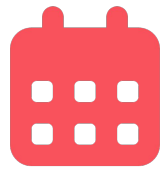


Source : [R for Data Science \(2e\)](#)



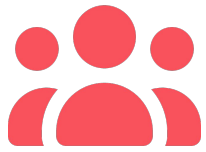
PROGRAMME MATINÉE

- **Introduction : la programmation, R et RStudio**, *comprendre les principes de la programmation, et les spécificités du langage R et de l'interface RStudio ;*
- **Importer des données ;**
- **Nettoyer et transformer des données :** *connaître les principales fonctions pour manipuler les données avec des exercices de mise en pratique ;*



PROGRAMME APRÈS-MIDI

- **Datavisualisation : comment faire de la visualisation sous R avec ggplot,** *appréhender la librairie de visualisation de données du tidyverse, avec des exercices de mise en pratique ;*
- **Cartographie : comment faire des cartes interactives en R,** *aperçu de la librairie leaflet ;*
- **Exercice final :** *reproduire une datavisualisation impliquant le traitement et la visualisation de données ;*
- **Conclusion de la journée :** vos retours, le mot de fin.



Avant de commencer

Faisons un rapide tour de table :

- qui êtes-vous ? 🔍
- sur une échelle de 1 à 3, à combien estimez-vous votre **niveau** en manipulation de données ? ★★☆☆
- sur une échelle de 1 à 3, à combien estimez-vous votre **intérêt** pour la manipulation de données ? ★★☆☆
- qu'attendez-vous de cette journée ? 💡

La programmation, R et RStudio

```
    }).done(function(response) {  
      for (var i = 0; i < response.length; i++) {  
        var layer = L.marker(  
          [response[i].latitude, response[i].longitude]  
          // , {icon: myIcon}  
        );  
        layer.addTo(group);  
  
        layer.bindPopup(  
          "<p>" + "Species: " + response[i].species + "<br>" +  
          "<p>" + "Description: " + response[i].description + "<br>" +  
          "<p>" + "Seen at: " + response[i].latitude + " " + response[i].longitude + "<br>" +  
          "<p>" + "On: " + response[i].sighted_at + "</p>"  
        );  
      }  
  
      $('select').change(function() {  
        species = this.value;  
      });  
    });  
  }  
  $.ajax({  
    url: queryURL,  
    method: "GET"  
  }).done(function(response) {  
    for (var i = 0; i < response.length; i++) {  
      var layer = L.marker(  
        [response[i].latitude, response[i].longitude]  
        // , {icon: myIcon}  
      );  
      layer.addTo(group);  
    }  
  });  
}
```



Quelques ressources à présenter

➤ les fiches tuto de POSIT



CHEATSHEET |

Data tidying with tidyr
cheatsheet

[https://posit.co/
resources/chea
tsheets/](https://posit.co/resources/cheatsheets/)

➤ un support complet pour approfondir les notions



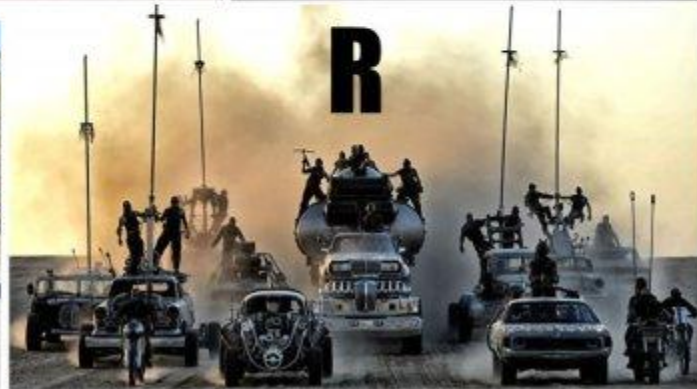
[https://dianethy.
github.io/cours
R/Introduction_R
.html](https://dianethy.github.io/cours_R/Introduction_R.html)

La **programmation** , tout un concept :

- **Définition** : ensemble des activités liées à la conception, l'écriture, le test et la maintenance de programmes informatiques à l'aide de langages de programmation (source : [wikipédia](https://fr.wikipedia.org/wiki/Programmation)).
- **Synonymes** : codage, développement.
- **Utilité** :
 - **autonomie** : pas de dépendance à un logiciel, permet d'automatiser des tâches répétitives
 - **scalabilité** : ce qui marche sur 100 lignes de données marche sur 100.000
 - **collaboration** : le code se partage facilement
 - **reproductibilité** : possible de refaire une analyse à l'identique
- **Usage** : de nombreux langages existent (HTML, Javascript, Java, C, Python, R...)

R, un langage de programmation **destiné** **aux statistiques**

If statistics programs/languages were cars...



Gratuit, puissant, design, avec une communauté importante ([Stack Overflow](#), [R-bloggers](#) etc.)

Les concepts clés :

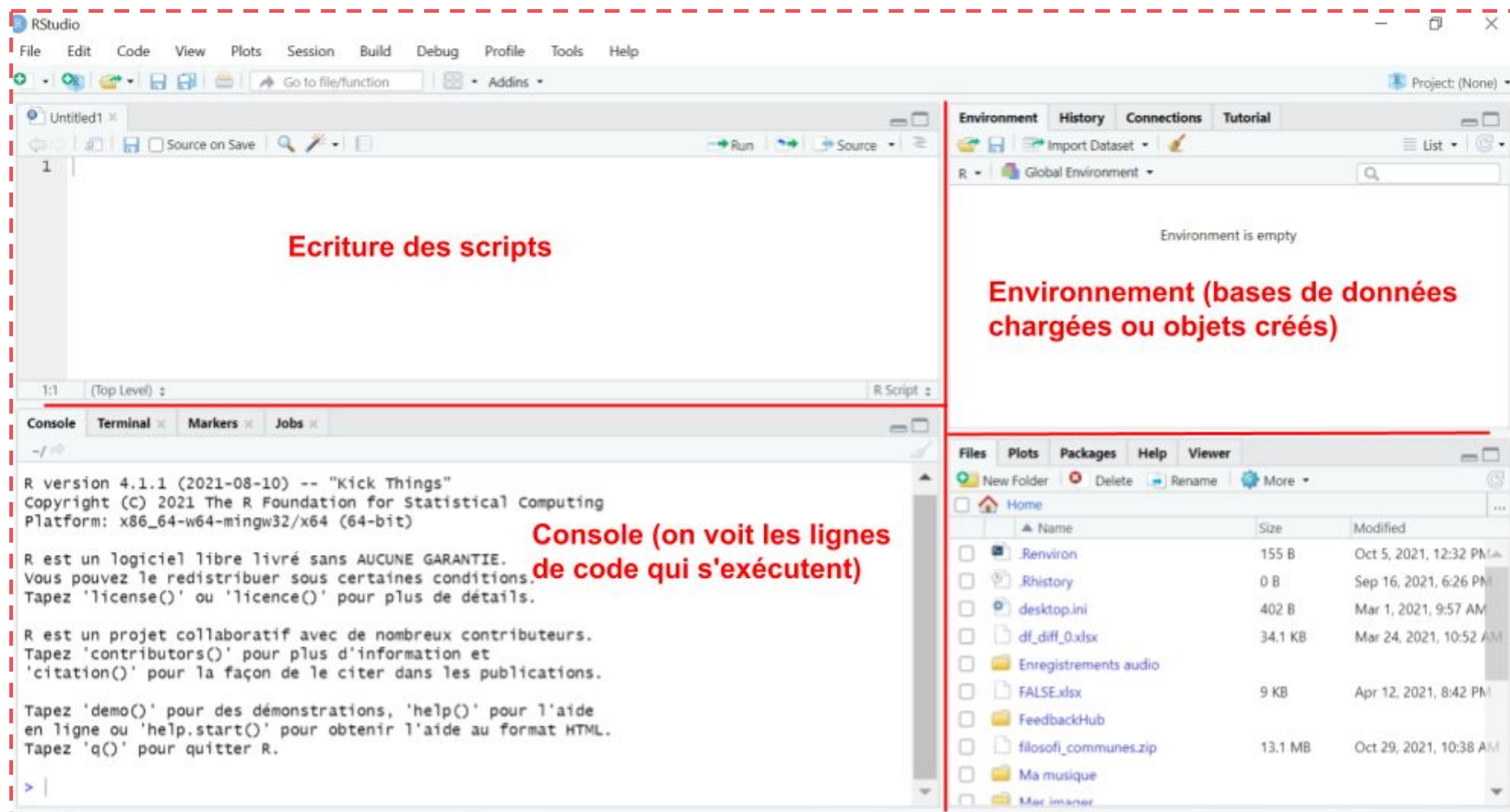
漢

- run
- package
- library
- script

Abc

- **exécuter** (une ligne de code)
- **ensemble de fonctions**
- **endroit où sont gardés les packages**
- **document où l'on code** (extension *.R*)

RStudio, une interface permettant d'interagir avec R



Assez parlé, ouvrons RStudio !

- Installez les packages dont nous aurons besoin aujourd'hui
 - `install.packages(c("readxl", "tidyverse", "janitor", "summarytools", "leaflet"))`
- Créez un nouveau script via: *File > New File > R script*
- Commencez à interagir par de simples opérations arithmétiques: $2+3$
- Exécutez les lignes de code: *Ctrl/Enter* ou bouton  Run
- Assignez des valeurs à des objets grâce à l'opérateur d'assignation `<-`
 - `x <- 2`
 - `y <- 7`
 - `resultat <- x * y`
 - `resultat`
 - `z <- "hello world"`
- Commentez le code via `#`

Deux manières de coder en R

- **R-base**

- exemple 1 : `exemple[,2]`
- exemple 2 : `f(g(h(exemple)))`

- **Tidyverse**

- exemple 1 : `exemple |> select(2)`
- exemple 2 : `exemple |> h() |> g() |> f()`



Le tidyverse fonctionne grâce au pipe (`%>%` ou `|>`), qui permet d'appliquer des fonctions à une base de données. L'opération à droite du pipe est appliquée à la valeur située à gauche du pipe, il devient alors possible d'enchaîner les traitements.

Récapitulatif des raccourcis clavier

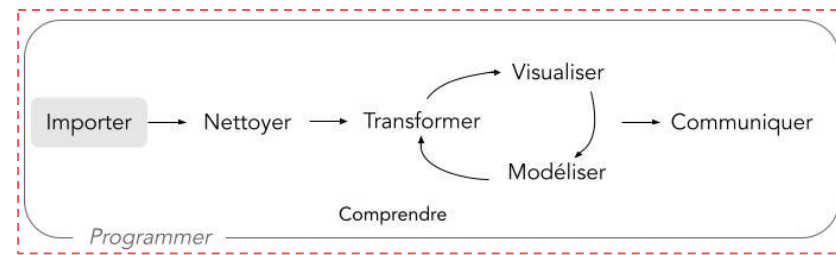
- pipe (%>% ou |>) : Ctrl, Maj, M
- opérateur d'assignation (<-) : Alt, -
- runner / exécuter une ligne : Ctrl, Enter
- ouvrir un nouveau script : Ctrl, Maj, N

Le tidyverse



Codons ensemble avec les principales fonctions du tidyverse

Importer des données



- **Excel**

- `library(readxl)`
- `exemple <- read_excel(path = "fichier.xlsx", sheet = "nom_feuille")`

- **CSV : valeurs séparées par des virgules**

- `library(readr)`
- `exemple <- read_csv("fichier.csv")`

- **SCSV : valeurs séparées par des points-virgules ("faux" CSV)**

- `library(readr)`
- `exemple <- read_delim("fichier.csv", delim = ";")`

💡 **ASTUCE :** si vous avez un doute, vous pouvez également importer les données en "press-bouton" via le Menu > File > Import Dataset >...

- **À vous !** Importer le jeu de données '**Prénoms donnés par département - Millésimé - France**' disponible sur OpenDataSoft :

- au format CSV en utilisant le lien [seulement en 1900 !!]
- si vous avez fini, au format tableur Excel en exportant le fichier [seulement en 1900 !!]

Importer des données

Solution

- Importer le jeu de données '**Prénoms donnés par département - Millésimé - France**' disponible sur OpenDataSoft :
 - en CSV en utilisant le lien

```
library(tidyverse)
data <-
read_delim("https://public.opendatasoft.com/api/explore/v2.1/catalog/datasets/demographyref-france-prenoms-departement-millesime/exports/csv?lang=fr&refine=birth_year%3A%221900%22&facet=facet(name%3D%22birth_year%22%2C%20disjunctive%3Dtrue)&timezone=Europe%2FBerlin&use_labels=true&delimiter=%3B",
, delim = ";")
```

- en excel en exportant le fichier

```
library(readxl)
data <-
read_excel("~/Downloads/demographyref-france-prenoms-departement-millesime.xlsx")
```



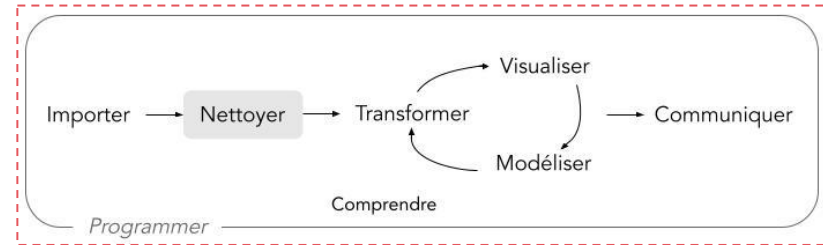
Astuce

Importer des données volumineuses

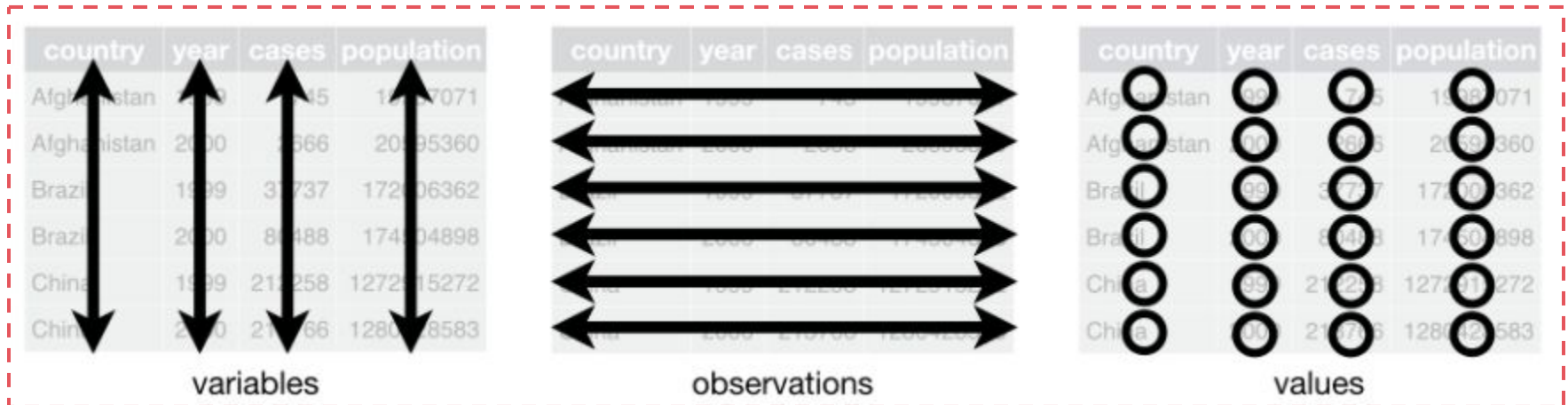
- La fonction `fread()` du package `data.table` permet d'importer des données volumineuses (>1GB) plus rapidement que les fonctions `readr`. Elle permet d'importer des formats CSV ou TSV, et on peut y spécifier les colonnes à importer pour limiter le volume des données.
 - `data.table::fread("dossier/mon_fichier_lourd.csv",
select = c("col1", "col4"))`
- Le package `data.table` contient de nombreuses fonctions pour manipuler de larges bases de données. La sémantique est cependant différente de celle que nous allons étudier aujourd'hui.

👉 [Lire la documentation complète de data.table](#)

Nettoyer des données



- **Structure de données propres**



- chaque ligne est une observation (entrée, enregistrement)
- chaque colonne est une variable
- une seule valeur dans chaque cellule

Nettoyer des données

- À vous !

- Quel jeu de données ci-dessous est proprement structuré ?
- Qu'est-ce qui ne va pas dans les 2 autres jeux ?

country	year	rate
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583

A.

country	year	key	value
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898
China	1999	cases	212258
China	1999	population	1272915272
China	2000	cases	213766
China	2000	population	1280428583

B.

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

C.

Nettoyer des données

Solution

- Quel jeu de données ci-dessous est proprement structuré ?
 - Le C.
- Qu'est-ce qui ne va pas dans les 2 autres jeux ?
 - Dans le A. on trouve 2 valeurs dans 1 case
 - Dans le B. les valeurs '*cases*' et '*population*' ne sont pas dans 2 colonnes séparées

Nettoyer des données

Avant toute analyse, il est nécessaire d'observer les données pour détecter les anomalies et les corriger

- **Types de données**

Famille de données	Structure	Nom retourné sous R	Exemple
<i>quantitatif</i>	entier	int = integer	3
	décimal inexact	dbl = double	3.4
	décimal exact	num = numeric	3.400001
<i>qualitatif</i>	chaîne de caractères	chr = character	Ville de Paris
	facteur à n niveaux	Factor	petit / moyen / grand
	facteur à 2 niveaux dit "booléen"	bool = boolean	0 / 1, True / False
<i>autre</i>	date	POSIX	13-12-1998

- **Pour les modifier**

```
as.integer()  
as.double()  
as.numeric()  
as.character()  
as.factor()  
as.logical()  
as.POSIXct()
```

- **Exemple de modification d'un typage de variable**

Si je veux mettre ma variable au format numérique : `as.numeric(ma_variable)`

Nettoyer des données

Avant toute analyse, il est nécessaire d'observer les données pour détecter les anomalies et les corriger

- **Les fonctions pour observer les données**
 - `glimpse(data)` : structure des données (on y voit tous les types)
 - `names(data)` : nom des colonnes
 - `view(dfSummary(data))` : tableau résumé des données
- **À vous !** Répondre aux questions suivantes :
 - le typage de la variable '*Code Officiel Région*' est-il correct ?
 - quel est le nombre de naissances maximal pour un prénom ?

Nettoyer des données

Solution

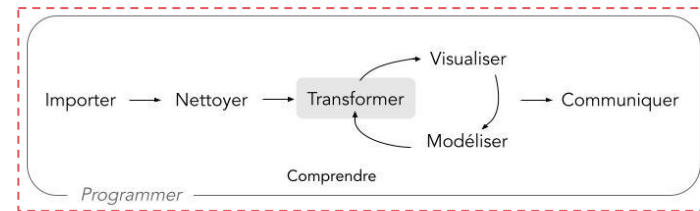
- le typage de la variable '*Code Officiel Région*' est-il correct ?

Le code officiel de la région correspond à un identifiant et non à un nombre à proprement parler (somme, taux, note etc.), donc doit être considéré comme une chaîne de caractères, qui correspond au format '*character*' sous R. Avec la fonction **glimpse(data)** on voit qu'il est considéré comme une chaîne de caractères ('*chr*'), donc son typage est **correct**.

- quel est le nombre de naissances maximal pour un prénom ?

La fonction **view(dfSummary(data))** nous informe que le maximum est de **2519**.

Transformer des données



Les 7 principales fonctions du **dplyr**, package de transformation de données contenu dans le **tidyverse**, sont les suivantes :

- **rename()** : pour renommer des colonnes
- **select()** et **relocate()** : pour sélectionner, supprimer ou changer l'ordre des colonnes
- **filter()** : pour sélectionner certaines lignes selon une condition
- **mutate()** : pour créer de nouvelles variables
- **summarise()** : pour résumer plusieurs valeurs en 1 valeur
- **group_by()** : pour grouper les observations avant d'appliquer une fonction

💡 **ASTUCE** : vous devrez maintenant utiliser le "pipe" (**|>** ou **|>**) pour appliquer ces fonctions. Vous pouvez l'écrire sur votre script grâce au raccourci clavier : Ctrl, Maj, M

Transformer des données : `rename()`

Cette fonction permet de renommer une ou plusieurs colonnes d'une base de données

- **Paramètres :**

- `names(data)` pour connaître le nom des colonnes des données
- `data <- data |> rename(nouveau_nom = `ancien nom`,
nv_nom = ancien_nom)`

- **Exemple :**

- `exemple <- exemple |> rename(tx_chomage = `Taux de chômage`,
regions = régions)`

- **À vous !**

- renommer la colonne 'Année de naissance' par 'annee_naissance'
- renommer la colonne 'Code Officiel Département' par 'code_departement'
- renommer la colonne 'Nom Officiel Département' par 'nom_departement'
- renommer la colonne 'Nombre de naissances' par 'nb_naissances'

Transformer des données : `rename()`

Solution

- renommer la colonne 'Année de naissance' par 'annee_naissance'
- renommer la colonne 'Code Officiel Département' par 'code_departement'
- renommer la colonne 'Nom Officiel Département' par 'nom_departement'
- renommer la colonne 'Nombre de naissances' par 'nb_naissances'

```
data <- data |>
  rename(annee_naissance = `Année de naissance`,
         code_departement = `Code Officiel Département`,
         nom_departement = `Nom Officiel Département`,
         nb_naissances = `Nombre de naissances`)
```



Astuce

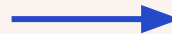
Renommer toutes les colonnes en une ligne de code

- La fonction `clean_names()` du package `janitor` applique automatiquement des règles pour transformer les noms de colonnes en `snake_case` en une seule opération

- `data |> janitor::clean_names()`

- Exemple :

Mon id	une_LETTRE	évènements
1	A	G
2	B	H



mon_id	une_lettre	evenements
1	A	G
2	B	H

Avant

Après



[Lire la documentation complète de janitor](#)

Transformer des données : `select()` et `relocate()`

La fonction `select()` permet de **sélectionner** ou **supprimer** certaines colonnes, par leur nom ou leur position dans la base de données.

La fonction `relocate()` permet de **changer l'ordre** des colonnes, par leur nom ou leur position dans la base de données.

- **Paramètres :**

- `data <- data |> select(col1, col3, col2, col4)`

- **Exemples :**

- `exemple <- exemple |> select(!col2)`

- `exemple <- exemple |> select(1:3, 7, 4:6)`

- **À vous !**

- **créer un nouvel objet** sans les variables *'annee_naissance'* et *'Année triable'*
- **créer un nouvel objet** avec toutes les variables de *'Sexe'* à *'nb_naissances'*
- **créer un nouvel objet** où *'code_departement'* est la première variable, en gardant les autres colonnes telles quelles

Transformer des données : `select()`

Solution

- **créer un nouvel objet** sans les variables *'annee_naissance'* et *'Année triable'*

```
dat_select1 <- data |> select(!annee_naissance, !`Année triable`)  
dat_select1 <- data |> select(!c(annee_naissance, `Année triable`))
```

- **créer un nouvel objet** avec toutes les variables de *'Sexe'* à *'nb_naissances'*

```
dat_select2 <- data |> select(Sexe:nb_naissances)
```

- **créer un nouvel objet** où *'code_departement'* est la première variable, en gardant les autres colonnes telles quelles

```
dat_relocate1 <- data |> relocate(code_departement)
```

Transformer des données : `filter()`

Cette fonction sert à sélectionner des lignes dans une base de données, répondant à certains critères ou conditions logiques

- **Paramètres :**

- `data <- data |> filter(col1 == "valeur")`
- opérations possibles : `==`, `!=`, `>`, `<`, `>=`, `<=`
- combinaisons d'opérations : `&` pour "et", `|` pour "ou"
- utiliser `nrow()` pour compter le nombre de ligne filtrées

- **Exemples :**

- `exemple <- exemple |> filter(col1 != 10 & col3 == "red")`
- `exemple |> filter(col > 0 | col1 <= 100) |> nrow()`

- **À vous !**

- **dans un nouvel objet**, filtrer les prénoms féminins
- **dans un nouvel objet**, filtrer les départements Aisne et Gironde

Transformer des données : `filter()`

Solution

- dans un nouvel objet, filtrer les prénoms féminins

```
dat_filter1 <- data |> filter(Sexe == "Féminin")
```

- dans un nouvel objet, filtrer les départements Aisne et Gironde

```
dat_filter2 <- data |> filter(nom_departement == "Aisne" |  
                             nom_departement == "Gironde")
```


Transformer des données : `mutate()`

Cette fonction permet de créer de nouvelles variables à partir des variables existantes, elle est très utile pour l'analyse de données

- **Paramètres :**

- `data <- data |> mutate(new_col = col1 * col2)`

- **Exemples :**

- `exemple <- exemple |> mutate(new_col = col1 / col2 * 100)`

- `exemple <- exemple |> mutate(new_col2 = col3 - 52)`

- `exemple <- exemple |> mutate(new_col2 = "Un nom à donner")`

- **À vous !**

- créer une **nouvelle variable** du nombre de naissances * 10

- créer une **nouvelle variable** de la part de chaque naissance (chaque ligne) sur toutes les naissances de 1900

Transformer des données : `mutate()`

Solution

- créer une **nouvelle variable** du nombre de naissances * 10

```
data <- data |> mutate(nb_naissances10 = nb_naissances * 10)
```

- créer une **nouvelle variable** de la part de chaque naissance (chaque ligne) sur toutes les naissances de 1900

```
data <- data |> mutate(taux = nb_naissances /  
sum(nb_naissances) *100)
```

Transformer des données : `summarise()`

Cette fonction permet de résumer plusieurs valeurs en une seule, ce qui est très utile pour avoir quelques statistiques sur les bases de données

- **Paramètres :**

- `data |> summarise(moyenne = mean(col1))`
- statistiques possibles : `mean`, `median`, `min`, `max`, `range`, `sum`, `n`
- si au moins une valeur manque (`NA`), ajouter l'argument `na.rm = TRUE`

- **Exemple :**

- `exemple |> summarise(min = min(col1, na.rm = TRUE),
max = max(col1, na.rm = TRUE))`

- **À vous !**

- calculer le nombre de naissances moyen et médian
- calculer l'étendue du nombre de naissances

Transformer des données : `summarise()`

Solution

- calculer le nombre de naissances moyen et médian

```
data |> summarise(moy = mean(nb_naissances),  
                  med = median(nb_naissances))
```

- calculer l'étendue du nombre de naissances

```
data |> summarise(etendue = range(nb_naissances))
```

Transformer des données : `group_by()`

Cette fonction permet de grouper les variables, elle est spécialement utile en analyse de données pour calculer des statistiques par groupe. Depuis janvier 2023, l'argument `.by` peut-être ajouté directement **dans** les fonctions du `dyplr`.

- **Paramètres :**

- ancienne manière de coder : `data |> group_by(group) |> summarise(min = min(col)) |> ungroup()`
- nouvelle manière de coder :
`data |> summarise(min = min(col), .by = group)`
- combiner avec les opérations déjà présentées : `mean`, `median`, `min`, etc

- **Exemple :**

- `exemple |> mutate(n = n(), .by = group_col)`

- **À vous !**

- calculer le nombre de naissances par sexe **dans un nouvel objet**
- calculer **dans une nouvelle variable de la base**, le nombre de prénoms (lignes) par région

Transformer des données : `group_by()`

Solution

- calculer le nombre de naissances par sexe **dans un nouvel objet**

```
group_data <- data |> summarise(naissances_sexe =  
sum(nb_naissances), .by = Sexe)
```

- calculer **dans une nouvelle variable de la base**, le nombre de prénoms (lignes) par région

```
data |> mutate(n_reg = n(), .by = `Nom Officiel Région`)
```

Une mise en pratique avant d'aller déjeuner



STATIONNEMENT – PLACE EN ZONE BLEUE

Exercice de mise en pratique

Vous avez 15 minutes pour effectuer ces quelques manipulations sur un jeu de données

- Depuis la base de données "**Stationnement – Place en zone bleue**" disponible sur le portail open data de La Rochelle :
 - quel est l'id de tronçon (pzb_troncon_id) du pzb_id '75' de la voie "Avenue des Corsaires" ?
 - combien y a-t-il de places de parking (lignes) sur le boulevard ANDRE SAUTEL ?

Exercice de mise en pratique

Solution

- Depuis la base de données "**Stationnement – Place en zone bleue**"

disponible sur le portail open data de La Rochelle :

- quel est l'id de tronçon (pzb_troncon_id) du pzb_id '75' de la voie "Avenue des Corsaires" ?

```
place_bleue <-  
read_csv("https://opendata.agglo-larochelle.fr/d4c/api/records/2.0/downloadfile/format=csv&resource_id=407dd9c7-5f0c-4f69-8746-ba2e1caaa291&use_labels_for_header=true&user_defined_fields=true")  
place_bleue |>  
  filter(pzb_id == 75 & pzb_voie == "Avenue DES CORSAIRES") |>  
  select(pzb_troncon_id)
```

- combien y a-t-il de places de parking (lignes) sur le boulevard ANDRE SAUTEL ?

```
place_bleue |> filter(pzb_voie == "Boulevard ANDRE SAUTEL") |> nrow()
```

Conclusion de la matinée

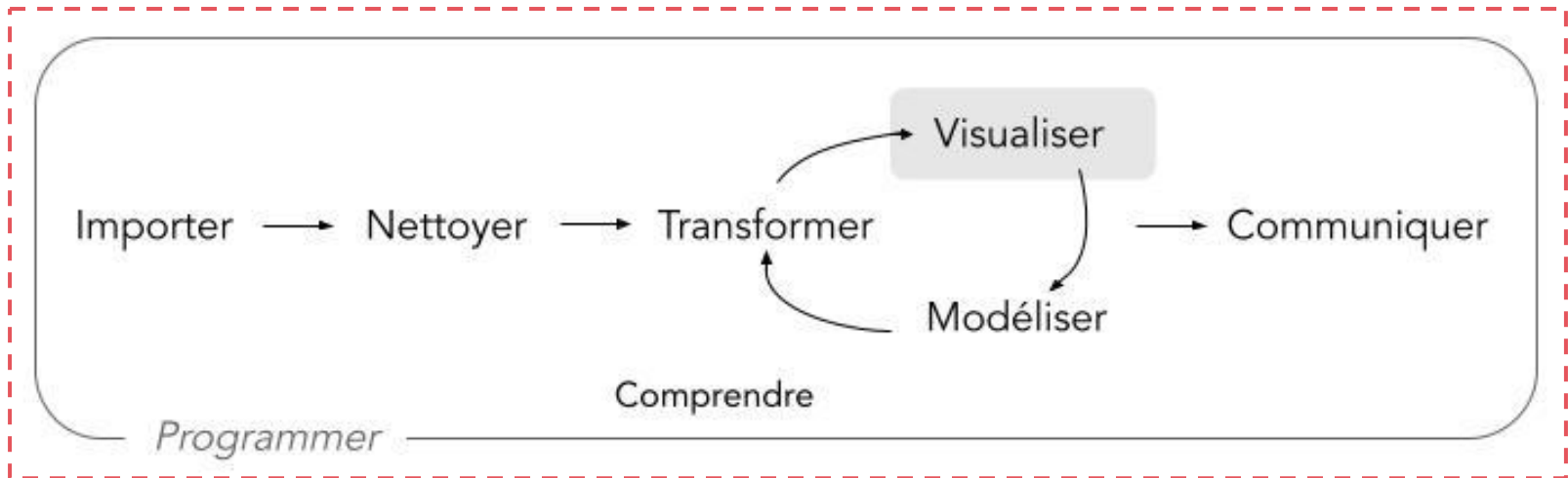


Avez-vous des questions ?

Résumons cette matinée

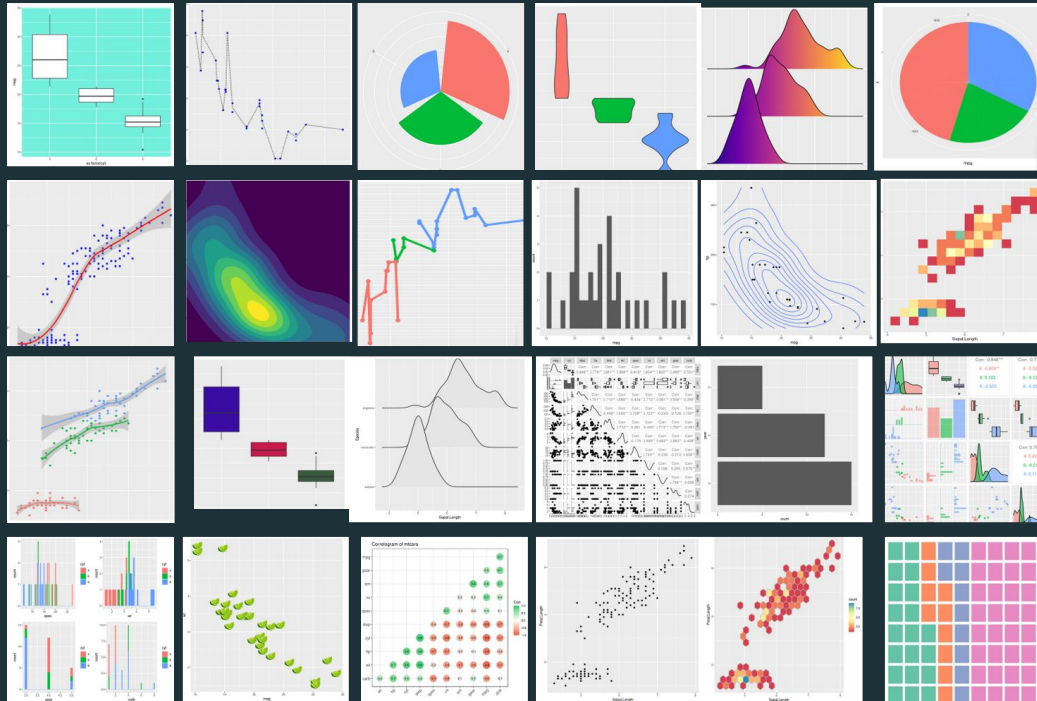
- Vous comprenez à quoi sert la programmation et pourquoi utiliser R via l'interface RStudio
- Vous avez fait vos premiers pas en code R :
 - import de données
 - nettoyage de données
 - transformation de données (`rename()`, `select()`, `relocate()`, `filter()`, `mutate()`, `summarise()`, `group_by()`)
- Vous avez mis en pratique toutes ces fonctions au fur et à mesure

Au programme de cette après-midi

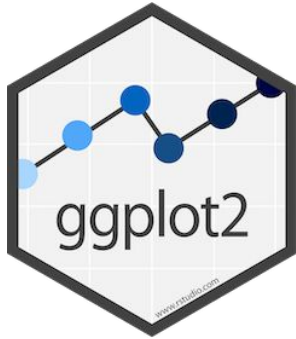


```
print("Bon appétit")  
[1] "Bon appétit"
```

Visualisation de données sous R

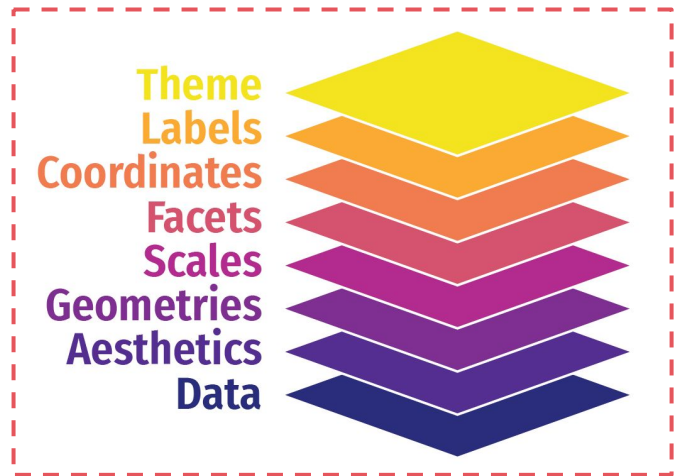


ggplot2, LA librairie de datavisualisation



La librairie **ggplot2** contenue dans le tidyverse permet de faire des graphiques sous R. Initialement développée par Hadley Wickham, cette librairie est l'une des plus connues du tidyverse.

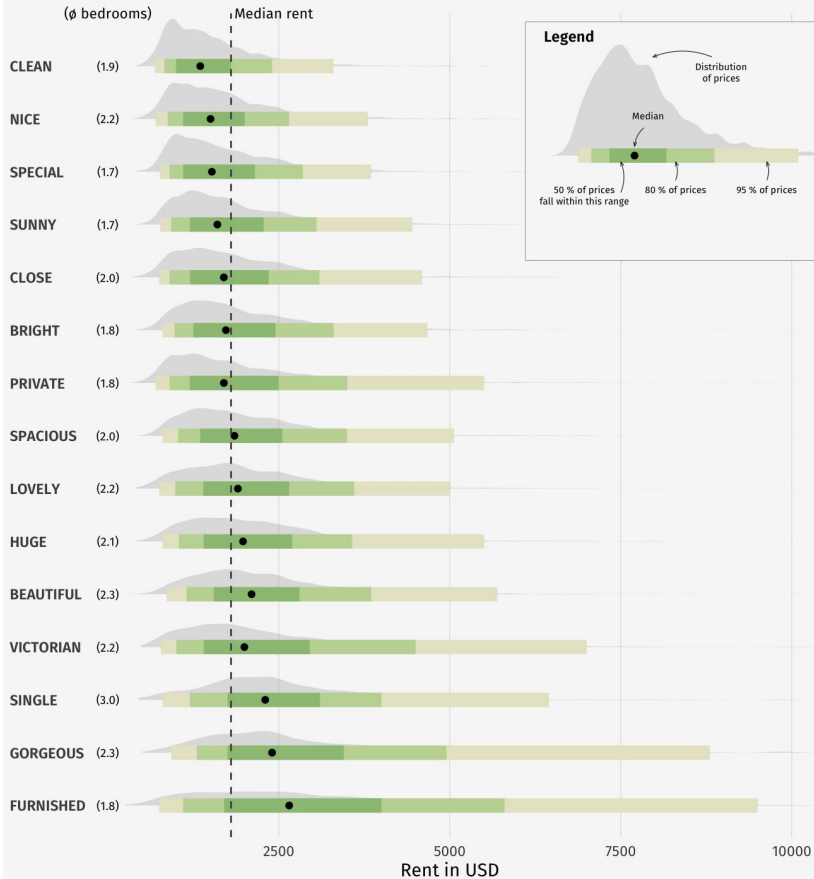
- **Sémantique des graphiques **ggplot2** :**
 1. **data** : les données à visualiser
 2. **aes** : les propriétés esthétiques
 3. **geom** : la géométrie/type de graphique
 4. **scale** : la gamme des valeurs
 5. **coord** : les coordonnées pour organiser la disposition
 6. **labels** : les étiquettes (noms)
 7. **facet** : les facettes pour sous-graphes
 8. **theme** : le thème appliqué aux éléments



Exemples de belles dataviz réalisées avec ggplot2

NICE AND CLEAN - RELATIVELY LOW RENT?

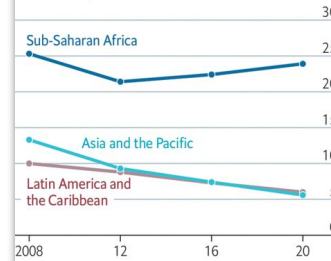
Adjectives used to describe houses and apartments in the San Francisco Bay Area in the titles of rental posts on Craigslist and how they are related to rental prices. Titles from 198,279 rental posts on Craigslist between 2000 and 2018. The 15 most frequent adjectives are shown.



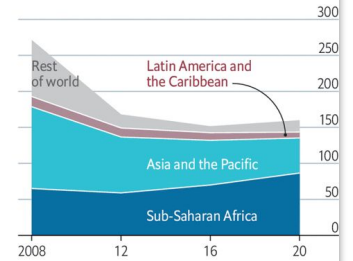
Axis capped at 10,000 USD.
Data: Pennington, Kate (2018). Bay Area Craigslist Rental Housing Posts, 2000-2018.
Retrieved from github.com/katepennington/historic_bay_area_craigslist_housing_posts/blob/master/clean_2000_2018.csv.zip.
Visualization: Ansgar Wolsing

All work, no play Children in child labour*

Selected regions, % of child population



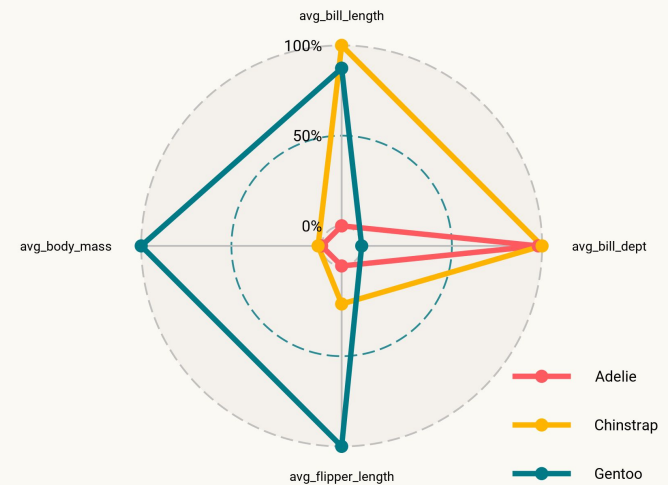
Number of children, m



Source: "Child Labour: Global estimates 2020, trends and the road forward", ILO and UNICEF

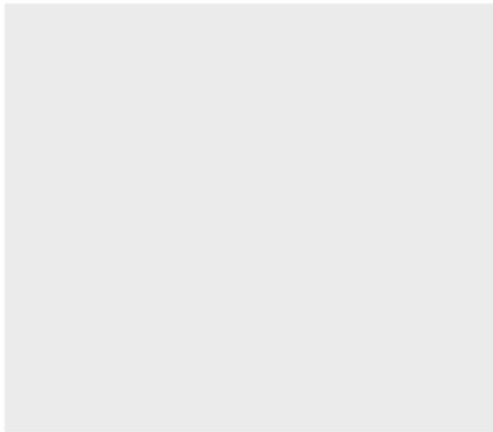
The Economist

Radar plot of penguins species

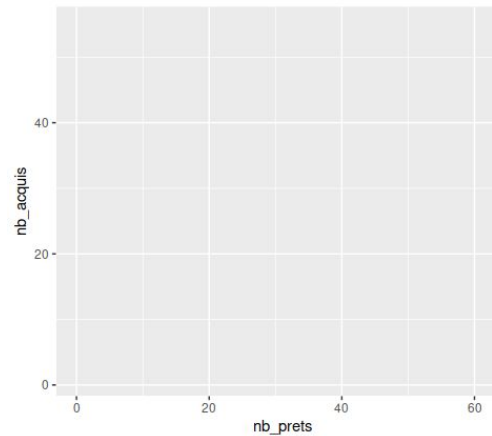


Le déroulé

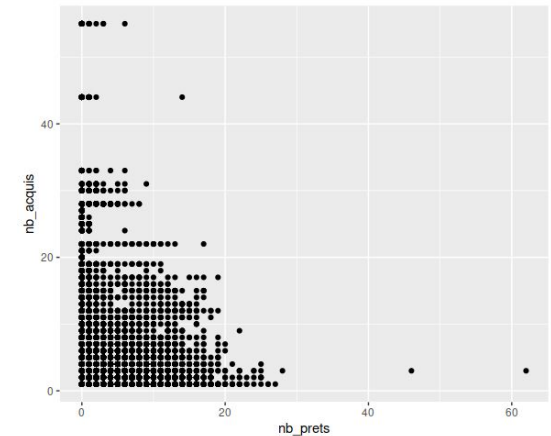
data : les données à visualiser



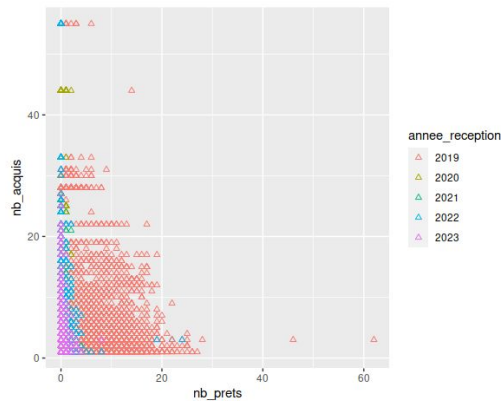
aes : les propriétés esthétiques



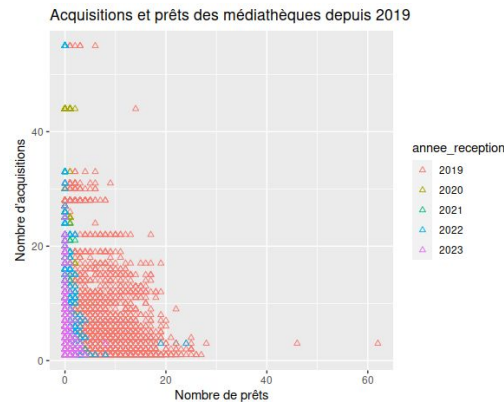
geom : la géométrie/type de graphique



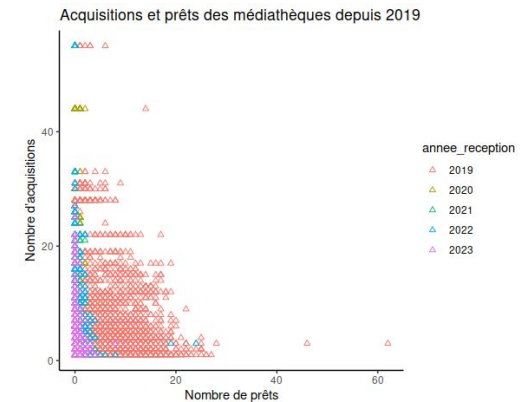
col, shape : les couleurs et les formes



labs : les noms et titres



theme : l'apparence



C'est parti !

Structuration de ggplot2 : data

Cet argument sert à appeler une base dont les données seront visualisées

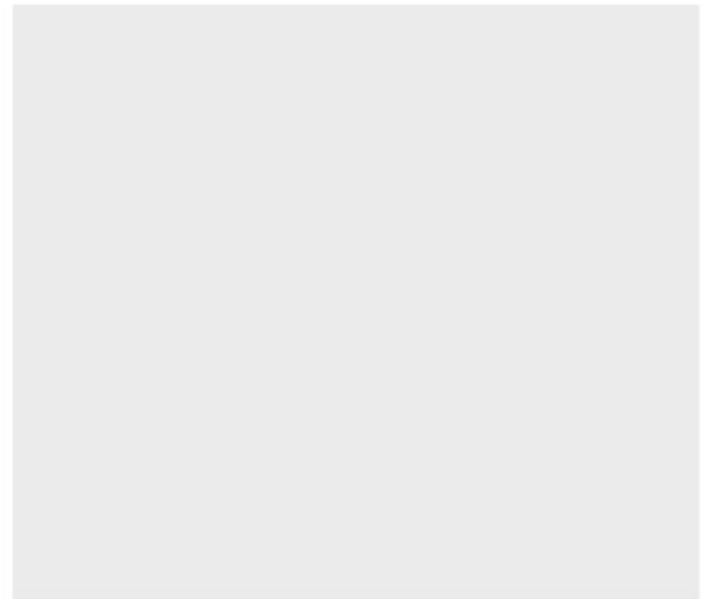
- **Paramètres :**
 - `ggplot(data = mes_datas)`
 - les prochains paramètres seront ajoutés au graphique par un `+` et non par le pipe `|>`
- **Exemple :**
 - `ggplot(exemple)` ou `exemple |> ggplot()`
- **À vous !**
 - exécuter le code de ce matin d'import et de manipulations des données de prénoms issues du portail OpenDataSoft
 - commencer un graphique ggplot2 sur la base de jeu de données créé "group_data"

Structuration de ggplot2 : data

Solution

- exécuter le code de ce matin d'import et de manipulations des données de prénoms issues du portail OpenDataSoft
- commencer un graphique ggplot2 sur la base de jeu de données créé "group_data"

```
ggplot(group_data)
```



Structuration de ggplot2 : aes()

Cet argument sert à indiquer les éléments et paramètres visuels qui dépendent de variables (on parle de *mapping*)

- **Paramètres :**

- `ggplot(data = mes_datas,
 mapping = aes(x = x, y = y))`
- **x** : abscisses, axe horizontal
- **y** : ordonnées, axe vertical

- **Exemple :**

- `ggplot(exemple, aes(x = col1, y = col2))`

- **À vous !**

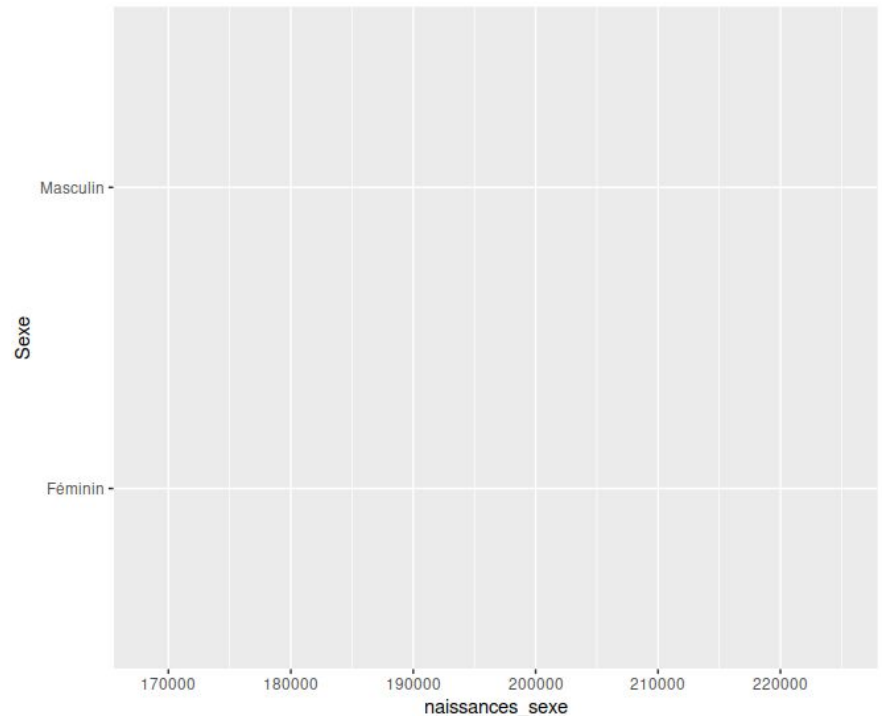
- continuer le graphique avec le nombre de naissances en abscisses et le sexe en ordonnées

Structuration de ggplot2 : `aes()`

Solution

- continuer le graphique avec le nombre de naissances en abscisses et le sexe en ordonnées

```
ggplot(group_data,  
  aes(x = naissances_sexe,  
      y = Sexe))
```



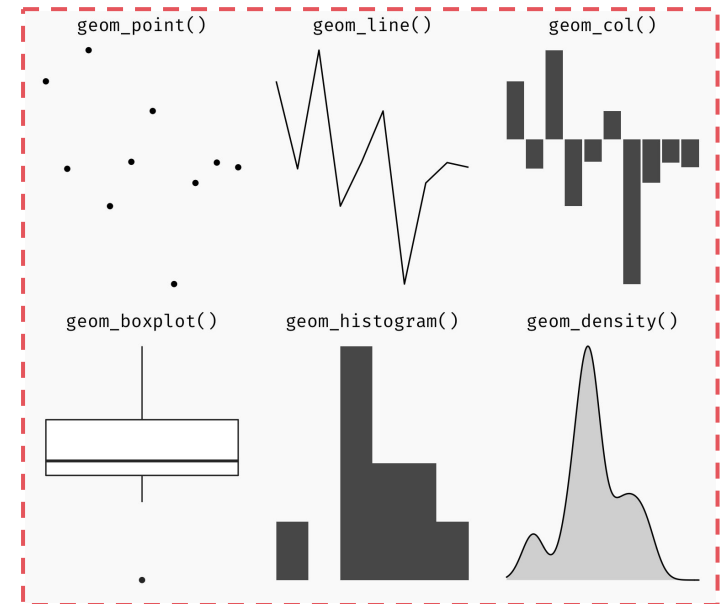
Structuration de ggplot2 : `geom_*`()

Cet argument sert à indiquer la géométrie utilisée, c'est-à-dire le type de datavisualisation

- **Paramètres :** `geom_*`()
 - `geom_point()` : nuage de points
 - `geom_line()` : ligne
 - `geom_boxplot()` : boîte à moustache
 - `geom_violin()` : violon
 - `geom_density()` : densité
 - `geom_histogram()` : histogramme
 - `geom_bar()` : barres

- **Exemple :**

- `ggplot(exemple,`
 `aes(x = col1, y = col2)) +`
 `geom_col()`



- **À vous !**

- continuer le graphique en ajoutant la géométrie 'barres'

Une différence subtile entre `geom_bar()` et `geom_col()`

`geom_col()` est un cas particulier de `geom_bar()` où `stat = "identity"`

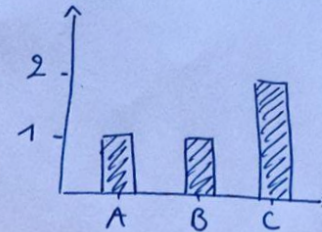
- Une illustration sur des données simples

X	Y
A	2
B	3
C	1
C	5

- Quelques explications :

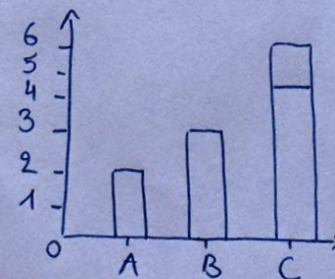
- par défaut `geom_bar()` visualise le **nombre de lignes** par catégorie de X
- en précisant `stat = "identity"` ou en utilisant `geom_col()`, ce sont **les valeurs d'une variable Y** qui sont représentées par la hauteur des barres

• `ggplot() + aes(x = X) +
geom_bar()`



• `ggplot() + aes(x = X, y = Y) +
geom_bar(stat = "identity")`

ou `ggplot() + aes(x = X, y = Y) +
geom_col()`



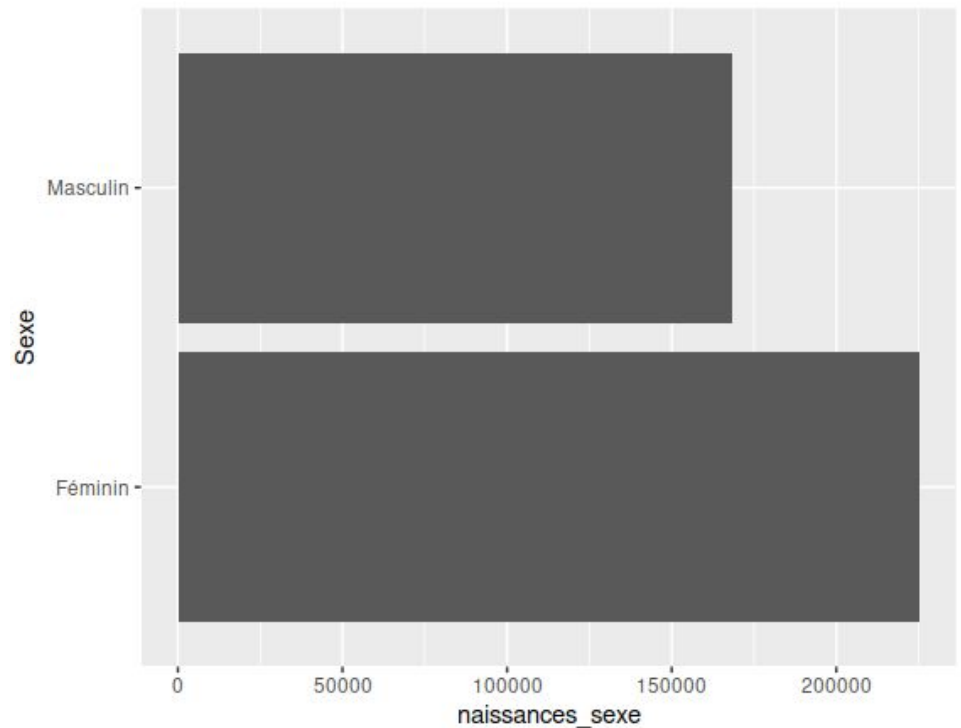
← va mettre toutes
les valeurs donc ici
1 et 5. Il faut passer
par une nouvelle variable
dans ce cas. `mutate()`

Structuration de ggplot2 : `geom_*()`

Solution

- continuer le graphique en ajoutant la géométrie 'barres'

```
ggplot(group_data,  
  aes(x = naissances_sexe,  
    y = Sexe)) +  
geom_col()
```



Structuration de ggplot2 : data, aes, geom

L'argument `aes()` peut être spécifié à différents endroits du code ; l'écriture est différente mais le résultat est identique

- **Différentes écritures :**

- `ggplot(exemple, aes(x = col1, y = col2)) +
 geom_col()`
- `ggplot(exemple) +
 geom_col(aes(x = col1, y = col2))`

Structuration de ggplot2 : data, aes, geom

Avec ces 3 couches essentielles, on peut "jouer" avec les paramètres pour améliorer les graphiques

- **Paramètres modifiables :**

- color, fill, alpha, size, shape, width...
- valeurs fixes ou dépendant de variables de la base de données

- **Exemple :**

- `ggplot(exemple,`
 `aes(x = col1, y = col2, width = col2)) +`
 `geom_col(col = "black", fill = "red")`

- **À vous !**

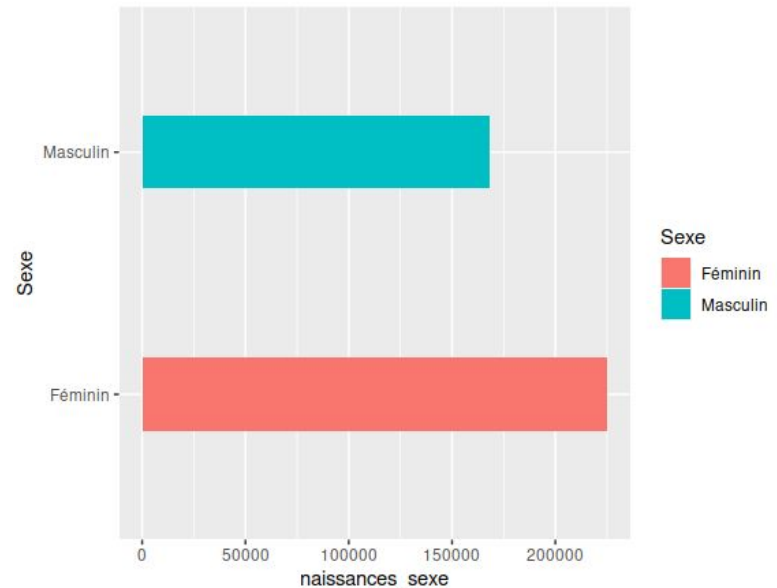
- continuer le graphique en colorant les barres selon le sexe, et en spécifiant la largeur des barres à 0.3

Structuration de ggplot2 : data, aes, geom

Solution

- continuer le graphique en colorant les barres selon le sexe, et en spécifiant la largeur des barres à 0.3

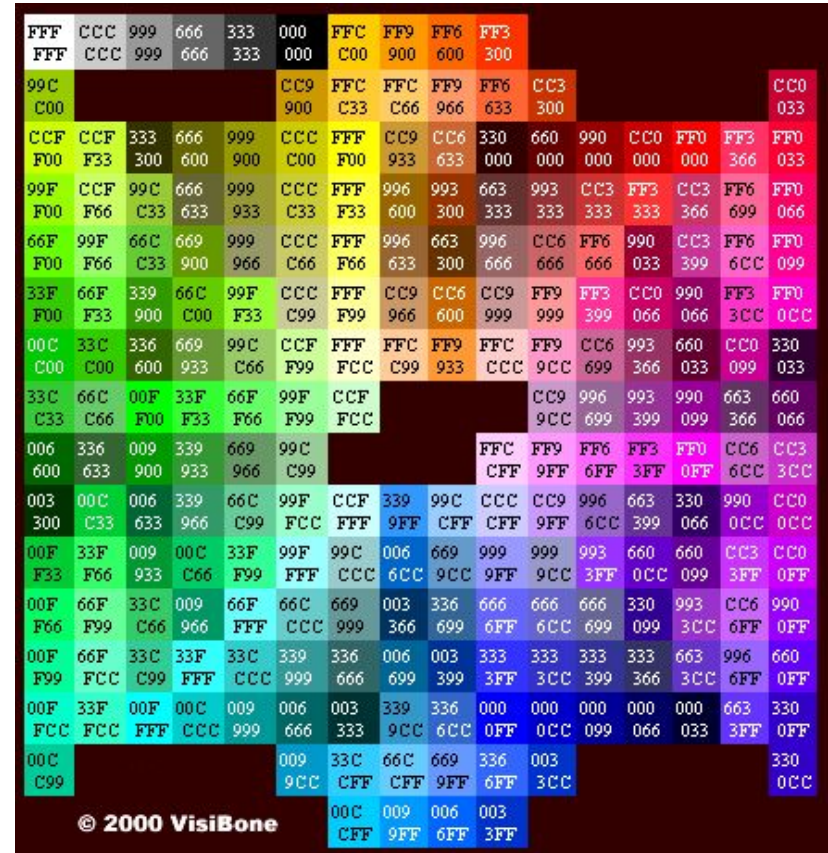
```
ggplot(group_data,  
  aes(x = naissances_sexe,  
      y = Sexe, fill = Sexe)) +  
  geom_col(width = 0.3)
```



Structuration de ggplot2 : colors

Les couleurs des graphiques sont importantes pour l'esthétique, la lisibilité, la compréhension, et le message à faire passer

- **2 manières de renseigner les couleurs :**
 - par son nom, ex: "red"
 - par son code HEX, ex: "#FFCC00"
- **Créer sa propre couleur :**
 - <https://www.color-hex.com/>



Structuration de ggplot2 : labs()

Cet argument sert à modifier les noms qui apparaissent sur le graphique

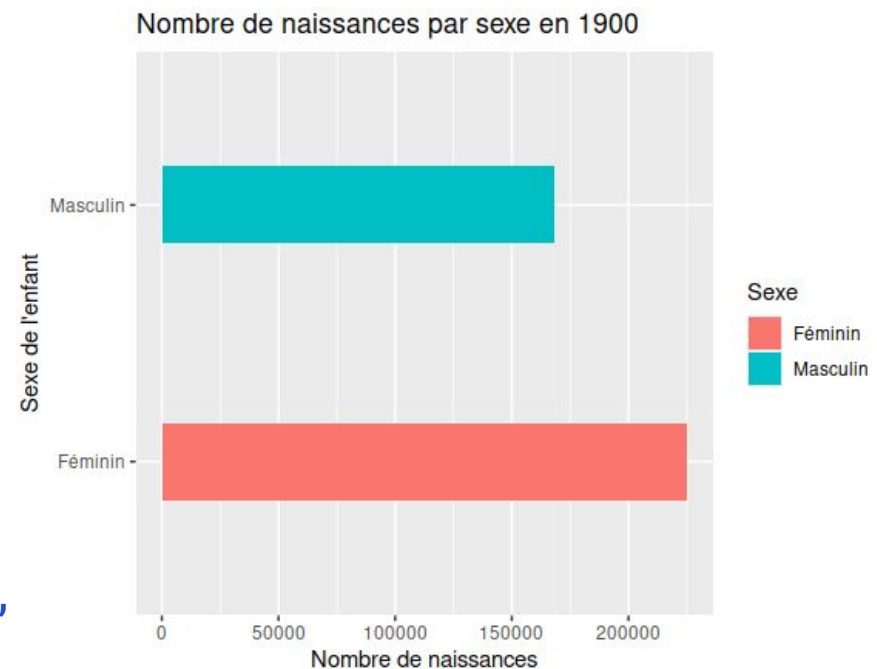
- **Paramètres :**
 - `labs(title, subtitle, caption, x, y)`
- **Exemple :**
 - ```
ggplot(exemple,
 aes(x = col1, y = col2)) +
 geom_histogram() +
 labs(title = "mon graphique", x = "Aa", y = "Bb")
```
- **À vous !**
  - continuer le graphique en nommant les axes X et Y, et en donnant un titre

# Structuration de ggplot2 : labs()

## Solution

- continuer le graphique en nommant les axes X et Y, et en donnant un titre

```
ggplot(group_data,
 aes(x = naissances_sexe,
 y = Sexe, fill = Sexe)) +
 geom_col(width = .3) +
 labs(x = "Nombre de naissances",
 y = "Sexe de l'enfant",
 title = "Nombre de naissances par sexe en 1900")
```



# Structuration de ggplot2 : `theme_*`()

Cet argument sert à modifier l'apparence du graphique (taille, polices, couleur des titres, arrière-plan, quadrillages etc.)

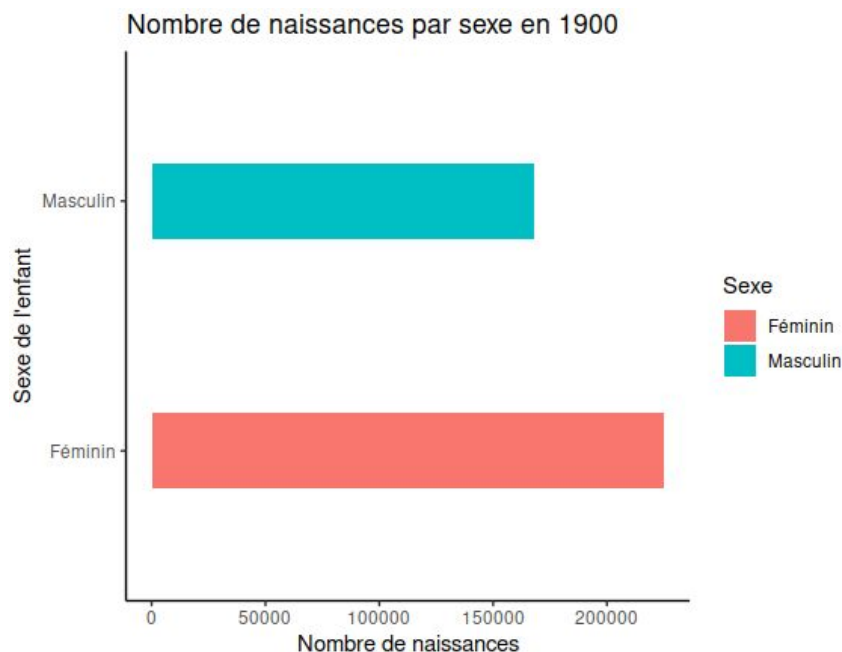
- **Paramètres :**
  - `theme_*`()
  - tous les thèmes prédéfinis dans ggplot2, consultables [ici](#)
- **Exemple :**
  - ```
ggplot(exemple, aes(x = col1, y = col2)) +  
  geom_histogram() +  
  theme_bw()
```
- **À vous !**
 - continuer le graphique en ajoutant le thème 'classic'

Structuration de ggplot2 : `theme_*`()

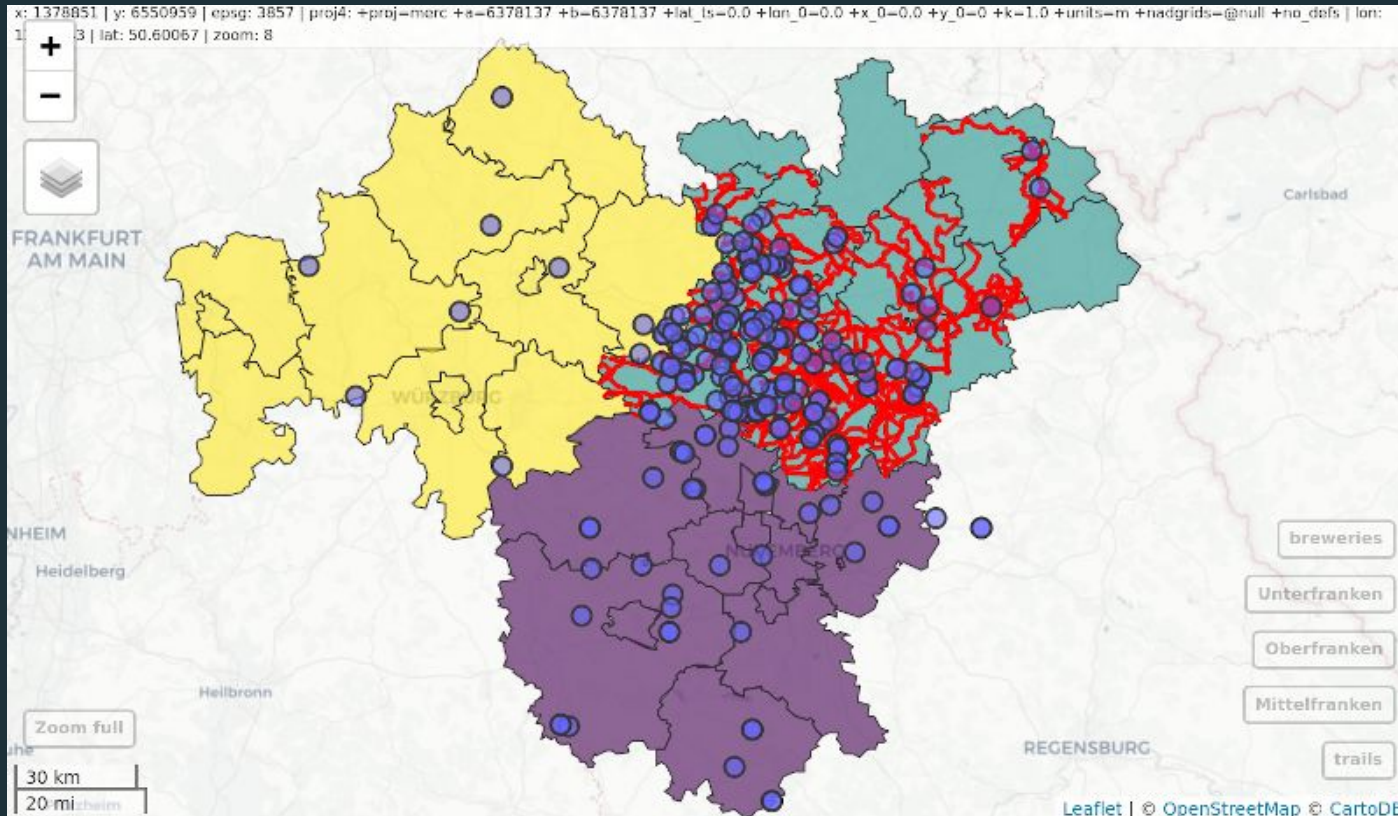
Solution

- continuer le graphique en ajoutant le thème 'classic'

```
ggplot(group_data,  
  aes(x = naissances_sexe,  
      y = Sexe, fill = Sexe)) +  
  geom_col(width = .3) +  
  labs(x = "Nombre de naissances",  
       y = "Sexe de l'enfant",  
       title = "Nombre de naissances par sexe en 1900") +  
  theme_classic()
```



La cartographie sous R



leaflet, une librairie de cartographie parmi de nombreuses existantes



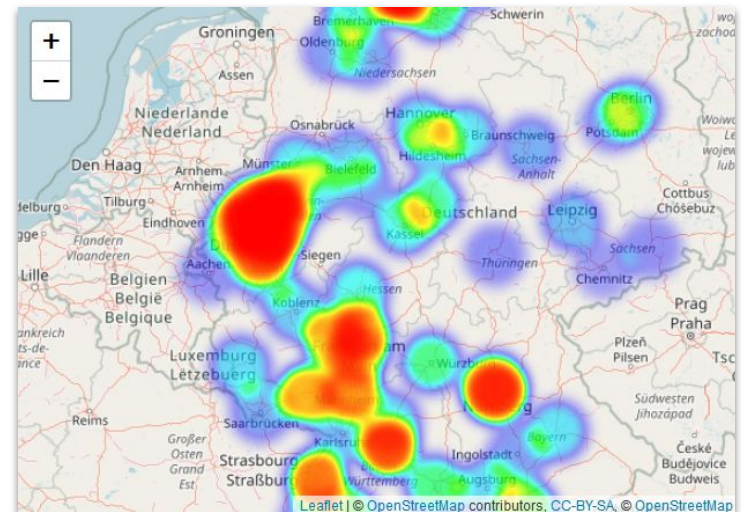
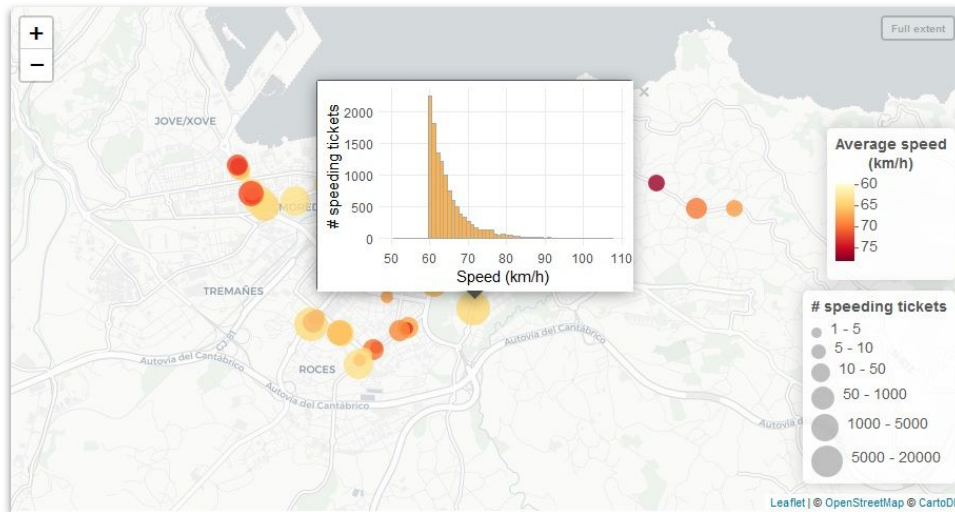
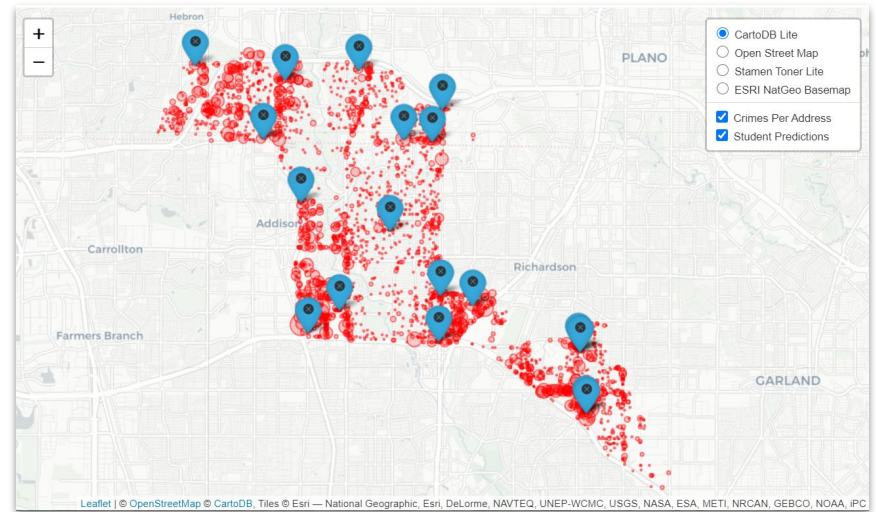
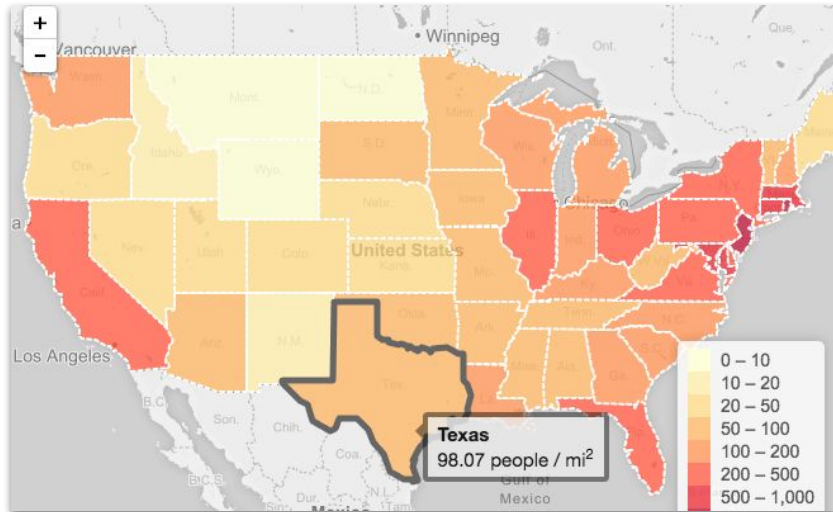
La librairie **leaflet** permet de représenter de manière **interactive** des données spatiales. Elle offre la possibilité de faire des cartes choroplèthes (avec à-plats de couleurs), des cartes à points (longitude/latitude) ou encore des cartes de chaleur.

- **Sémantique des cartographies **leaflet** :**
 1. **leaflet()** : ouvre un objet carte
 2. **addTiles()** : met un fond de carte
 3. **add*()** : ajoute un élément sur la carte
 4. **color, opacity, label** : options pour personnaliser la carte



👉 [Lire la documentation complète de leaflet](#)

Exemples de belles cartes réalisées avec leaflet



Le déroulé

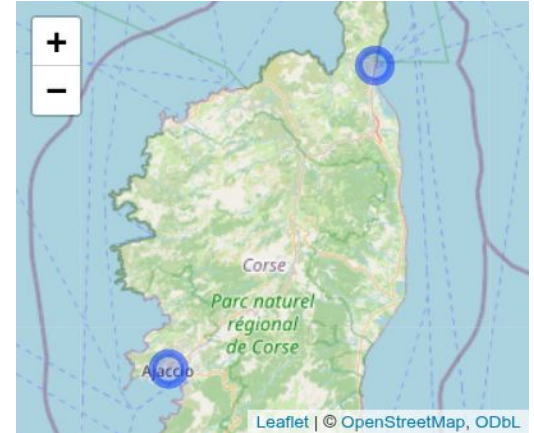
`leaflet()` : ouvre un objet carte



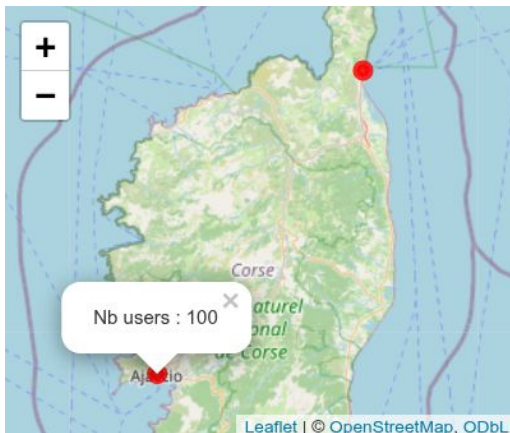
`addTiles()` : met un fond de carte



`add*()` : ajoute un élément sur la carte



`color`, `opacity`, `radius...` : paramètres pour personnaliser la carte



Structuration de leaflet : `leaflet()`

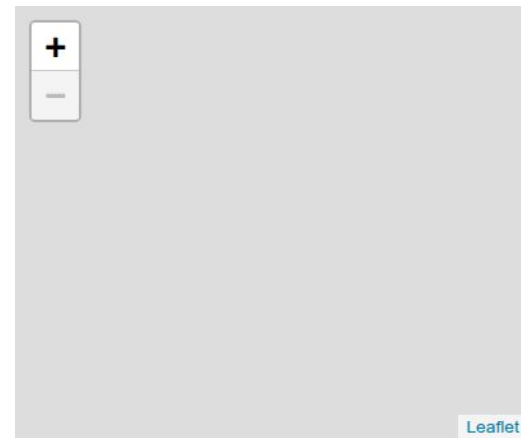
Ce premier élément sert à démarrer un objet de cartographie leaflet

- **Paramètres :**

- `leaflet()`
- les prochains paramètres seront ajoutés la carte par le pipe `|>`

- **Code**

```
leaflet()
```



Structuration de leaflet : `addTiles()`

Ce deuxième élément sert à ajouter un fond de carte

- **Paramètres :**

- `leaflet() |> addTiles()`
- par défaut, c'est le fond de carte OpenStreetMap qui est chargé
- d'autres fonds de carte peuvent être utilisés via

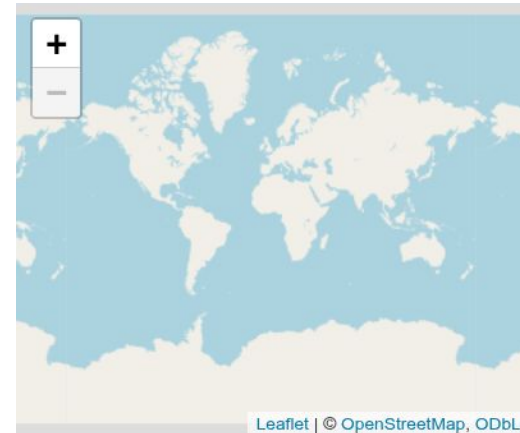
`leaflet() |> addTiles() |> addProviderTiles("nom_tile")`

exemple : `addProviderTiles(provider = "Esri.WorldGrayCanvas")`

👉 [voir l'ensemble des fonds de carte](#)

- **Code**

```
leaflet() |> addTiles()
```



Structuration de leaflet : `add*()`

Ce troisième élément sert à ajouter un marqueur, un point ou un polygone sur la carte

- **Différents éléments spatiaux à ajouter :**

- marqueur : `addMarkers()`
- cercle : `addCircles()` ou `addCircleMarkers()`
- polygone : `addPolygons()`

→ ces éléments peuvent être combinés les uns avec les autres

- **Code**

```
leaflet() |>  
  addTiles() |>  
  addMarkers(lng = -3.7, lat = 40.4)
```



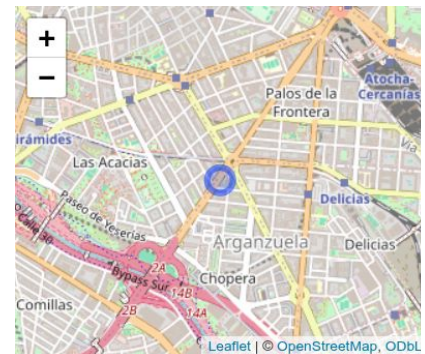
marqueur

```
leaflet() |>  
  addTiles() |>  
  addCircles(lng = -3.7, lat = 40.4)
```



cercle fixe

```
leaflet() |>  
  addTiles() |>  
  addCircleMarkers(lng = -3.7, lat = 40.4)
```



**cercle de 500m
autour d'un point**


```

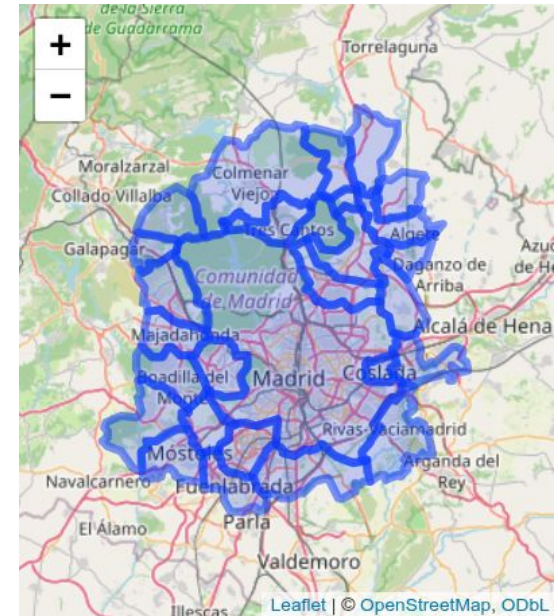
library(osmdata)
library(sf)
library(leaflet)

madrid_osm <- opq(bbox = "Madrid, Spain") |>
  add_osm_feature(key = "boundary",
                  value = "administrative") |>
  add_osm_feature(key = "admin_level",
                  value = "8") |>
  osmdata_sf()

madrid_boundary <- madrid_osm$osm_multipolygons |>
  st_make_valid()

leaflet() |>
  addTiles() |>
  addPolygons(data = madrid_boundary)

```



polygonos

Structuration de leaflet : color, opacity, radius...

Ces paramètres servent à personnaliser les éléments spatiaux ajoutés à la carte

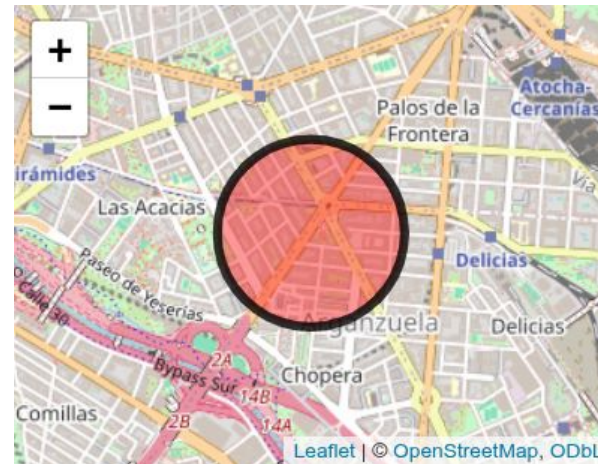
- **Différents paramètres peuvent être modifiés, voici les principaux :**

- couleurs : `color` et `fillColor`
- opacité : `opacity` et `fillOpacity`
- taille : `radius`

→ ces paramètres peuvent être modifiés tous à la fois et il en existe bien d'autres (à retrouver dans les documentations leaflet).

- **Code**

```
leaflet() |>  
  addTiles() |>  
  addCircleMarkers(lng = -3.7, lat = 40.4, radius = 50,  
                   color = "black", opacity = 0.8,  
                   fillColor = "red", fillOpacity = 0.4)
```



Des questions ?

Exercice final à choisir



Dataviz à reproduire - Cartographies des places de stationnement

Vous avez 30 minutes pour reproduire la carto suivante, en transformant et visualisant les données "*Stationnement - places disponibles en temps réel des parkings*" du portail open data de La Rochelle



- Au survol de la souris, le nom de la station s'affiche
- La taille des points varie selon le % de places disponibles
- Vous n'aurez pas les mêmes valeurs car les données sont mises à jour tous les jours (celles-ci sont du 24/06)

Dataviz à reproduire - Cartographies des places de stationnement

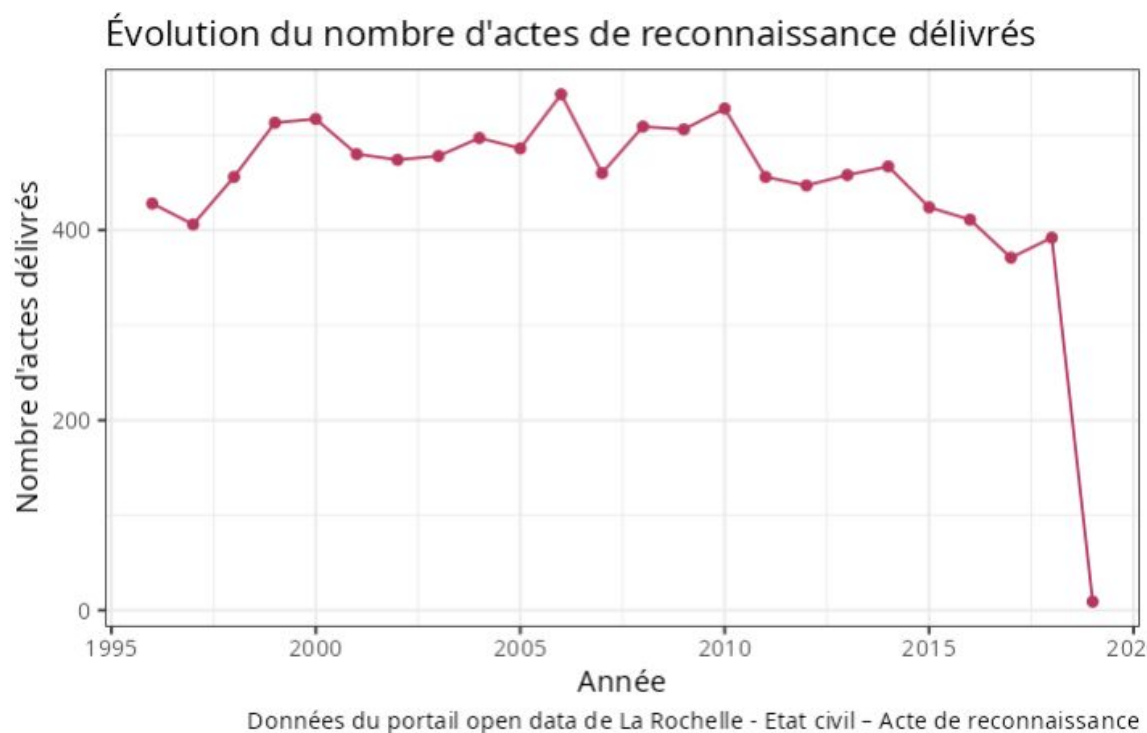
Vous avez 30 minutes pour reproduire la carto suivante, en transformant et visualisant les données "*Stationnement - places disponibles en temps réel des parkings*" du portail open data de La Rochelle

Indications et conseils :

- la première étape consiste évidemment à **importer les données** du portail (le format CSV convient puisqu'on n'a pas besoin de géométries qui requièrent un format spatial, seulement de la longitude / latitude)
- la deuxième étape est de **calculer le taux de places disponibles** (à partir de *nb_places* et *nb_places_disponibles*)
- la troisième étape est de **cartographier le tout** en jouant sur les différents paramètres
 - si la taille des points vous paraît trop petite ou trop grande, n'hésitez pas à l'ajuster en divisant ou multipliant sa valeur

Dataviz à reproduire - Évolution du nombre d'actes de reconnaissance délivrés

Vous avez 30 minutes pour reproduire le graphique suivant, en transformant et visualisant les données "*Etat civil – Acte de reconnaissance – Par année et par mois*" du portail open data de LR



- Le graphique présente un point de données par année
- Il superpose 2 géométries différentes

Dataviz à reproduire - Évolution du nombre d'actes de reconnaissance délivrés

Vous avez 30 minutes pour reproduire le graphique suivant, en transformant et visualisant les données "*Etat civil – Acte de reconnaissance – Par année et par mois*" du portail open data de LR

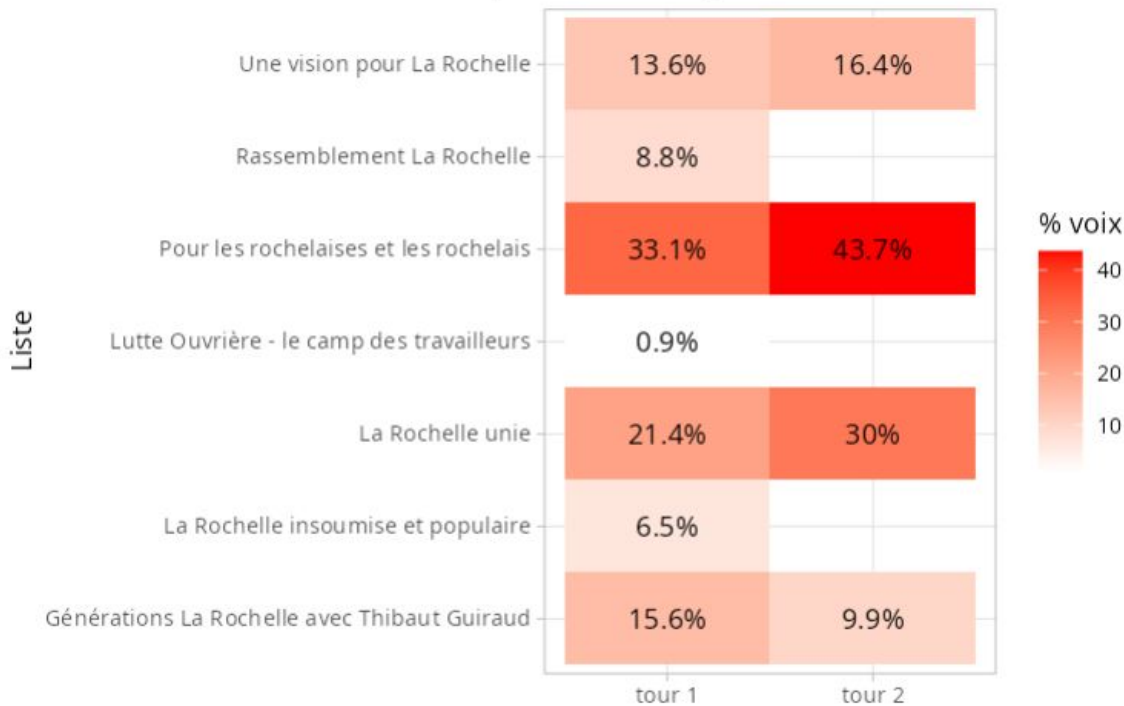
Indications et conseils :

- la première étape consiste évidemment à **importer les données** du portail (le format CSV convient)
- la deuxième étape est de **calculer le nombre d'actes délivrés par année** qui sont actuellement détaillés par année et par mois
- la troisième étape est de **visualiser le tout** en jouant sur les différents paramètres

Dataviz à reproduire - Heatmap des résultats aux municipales 2026

Vous avez 30 minutes pour reproduire la heatmap suivante, en transformant et visualisant les données "*Elections - Municipales 2026 Ville de La Rochelle - ** tour - ensemble de la ville*" du portail open data de LR

Résultats aux élections municipales de 2026, La Rochelle



Données du portail open data de La Rochelle - Elections - Municipales 2026 Ville de La Rochelle

- Le graphique présente un point de données par année
- Il superpose 2 géométries différentes

Dataviz à reproduire - Heatmap des résultats aux municipales 2026

Vous avez 30 minutes pour reproduire la heatmap suivante, en transformant et visualisant les données "*Elections - Municipales 2026 Ville de La Rochelle - ** tour - ensemble de la ville*" du portail open data de LR

Indications et conseils :

- la première étape consiste évidemment à **importer les données** du portail (le format CSV convient), vous aurez alors 2 bases de données distinctes ; une pour le premier tour, une pour le second
- la deuxième étape est de **rassembler les 2 bases en une**. Vous pourrez le faire grâce à la fonction `bind_rows()` ([voir la doc](#)). Pensez à créer une variable qui vous permette d'identifier le tour avant de rassembler les données
- la troisième étape est de **visualiser le tout** en une heatmap (carte de chaleur) grâce à la fonction `geom_tile()` du package ggplot. Pour ajouter du texte sur le graphique, regardez la fonction `geom_text()`

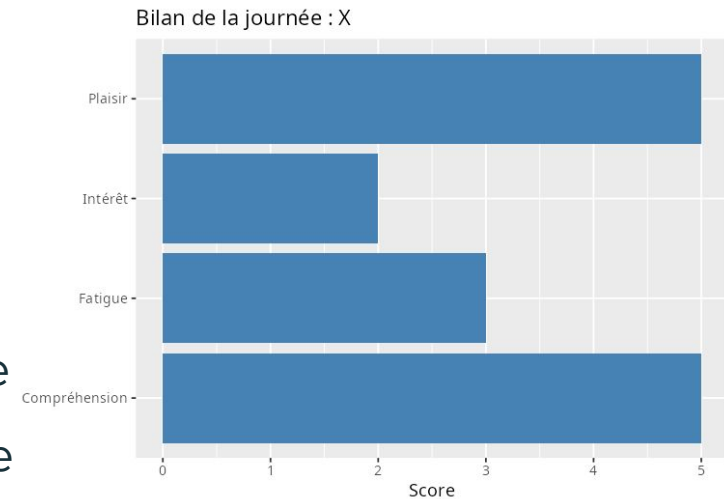
Conclusion

Résumons cette après-midi

- **ggplot2**, vous avez appris à construire un graphique pas à pas sur R, avec la librairie de datavisualisation
 - les incontournables : `data`, `aes()`, `geom_*()`
 - les plus : `colors`, `labs()`, `theme_*()`...
- **leaflet**, vous avez appris à construire une cartographie pas à pas sur R
 - les incontournables : `leaflet()`, `addTiles()`, `add*()`
 - les plus : `color`, `opacity`, `radius`...
- **Reproduire une dataviz**, vous avez transformé puis visualisé les données de *opendata.agglo-laroche.fr*, dans le but de s'approcher le plus possible du modèle de dataviz proposé

Bilan de cette matinée

- À partir du code ci-dessous :
 - **copiez-le et collez-le** dans RStudio
 - **adaptez** la ligne '*valeur*' selon votre journée
 - **exécutez** le code pour générer le graphique
 - **personnalisez** si vous le souhaitez
- **Faites une capture d'écran** du graphique et **déposez-la** sur la slide suivante

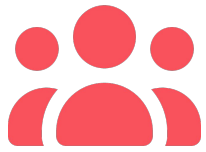


```
library(tidyverse)
```

```
data_barplot <- tibble(categorie = c("Compréhension", "Intérêt", "Fatigue",  
                                     "Plaisir"),  
                       valeur = c(5, 2, 3, 5))
```

```
data_barplot |>  
  ggplot(aes(x = categorie, y = valeur)) +  
  geom_col(fill = "steelblue") +  
  labs(title = "Bilan de la journée : X", x = NULL, y = "Score") +  
  coord_flip()
```

Vos bilans !



Pour terminer

Un tour de table pour entendre vos retours :

- qu'allez-vous faire de tout ça ?
- pensez-vous que la programmation puisse vous aider ?
- si oui, dans quelles tâches ?

Bibliographie

Des documentations pour approfondir

:

- **Cheat sheets**, "antisèches" rendant facile l'apprentissage et l'utilisation de packages R (*recharger les pages si les PDF ne s'affichent pas*) :
 - import de données, nettoyage de données, transformation de données, visualisation de données avec ggplot2
- **R for Data Science**, livre incontournable pour apprendre la science de données sous R, écrit par *Hadley Wickham* et *Garrett Grolemund* ;
- **R Graph Gallery**, aide et inspiration pour réaliser des graphiques sous R ;
- **Qu'est-ce que la data visualisation, ou dataviz ? Définition et exemples** ;
- **What would you like to show**, schéma récapitulatif de la visualisation à utiliser en fonction du type de données représenté ;

Des documentations pour approfondir

:

- [Visual vocabulary](#), poster des types de graphiques.
- [tidygeocoder](#), un package pour géocoder (passer d'une adresse à sa longitude / latitude, et inversement) ;
- [leaflet](#), un package pour cartographier des données sous R.

Merci pour votre attention !

diane@dataactivist.coop